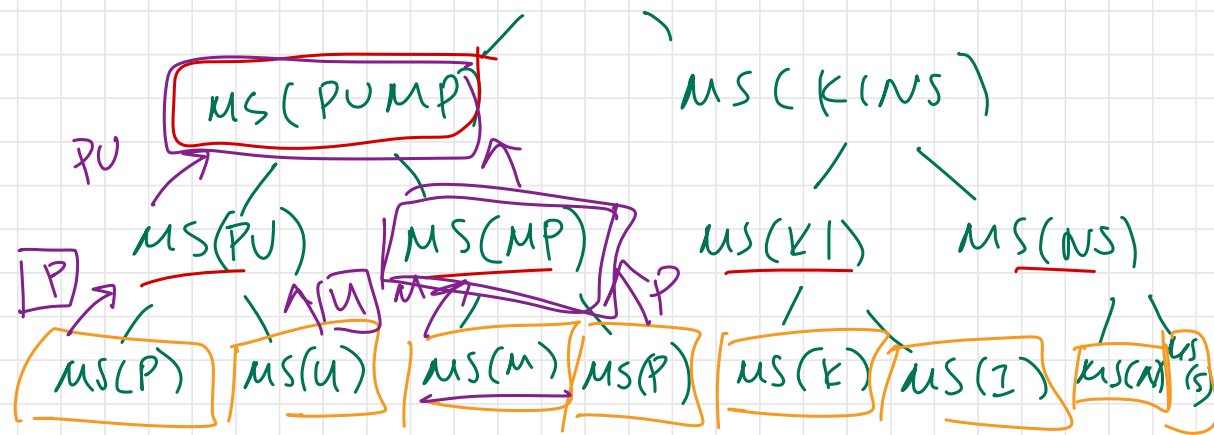


w/ table:

- write the pseudocode for mergesort
- you can call merge (sorted arrays B, C)
- if done: draw tree of MS calls on PUMPKINS

MS(PUMPKINS)



^{MS}
mergeSort (array A of length n):

if $n > 1$:

$S_L = \text{mergeSort}(\text{L half of A})$

$S_R = \text{mergeSort}(\text{R half of A})$

return merge(S_L, S_R)

else:

return A

$T(n)$ = runtime of mergesort on an array of length n

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

recurrence

2 calls to MS on arrays of length $n/2$

one call to merge on arrays of length $n/2$

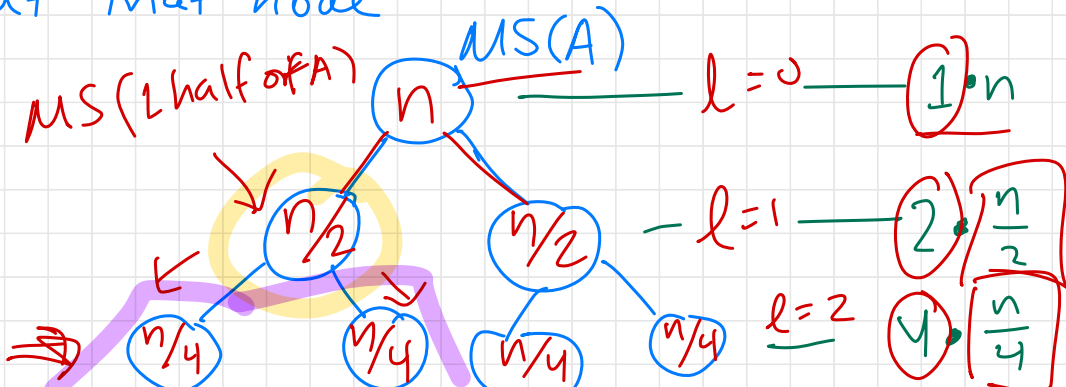
Third MS: recursive runtime?

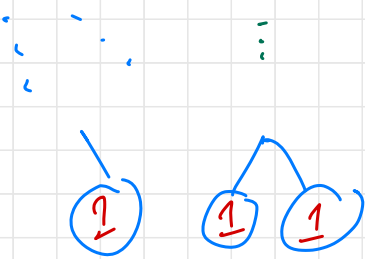
$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T(n/3) + \Theta(n) & \text{if } n > 1 \end{cases}$$

???

Recursion Tree

- rooted trees w/ one node per call
- to find closed form of $T(n)$, value of node is non-recursive work done at that node





→ sum of sums of all layers
each level contributes n
how many levels?

2^l nodes on each level

what is work^{node} at level l ?

2x $l=3$, $??$ $\frac{n}{2^l}$